

Summer Term 2026
Computer Animation - Assignment Sheet 4
Due Date, June 8, 2026

Assignment 1 (*Motion Capture Filtering, 12 Points*)

In this assignment, you will work with motion capture data in ASF/AMC format. The ASF file contains the skeleton structure, and the AMC file contains the recorded motion frames. Together with this exercise sheet, you receive a Python handout containing the following components:

1. a parser for ASF/AMC motion capture data,
2. a 3D viewer for visualizing one or more motion sequences,
3. starter scripts for this assignment,
4. the files `HDM_bk.asf` and `HDM_bk.walk.amc`.

You may also download and test additional motion files from the HDM05 database:
<http://www.mpi-inf.mpg.de/resources/HDM05/>

Use the provided Python code to load the skeleton and motion data:

```
joints = parse_asf("data/HDM_bk.asf")
motions = parse_amc("data/HDM_bk_walk.amc")
```

You can visualize a motion sequence using the viewer:

```
viewer = Viewer(joints=(joints,), motions=(motions,), legends=("Original",))
viewer.run()
```

Read through the provided README.md file for more information.

Your task is to apply a low-pass filter to the motion data and compare the results visually. Use different window sizes, for example 32, 64, 128 and 256 frames. You should filter and compare the motion on the following levels:

1. world-space joint positions,
2. local Euler-angle rotations,
3. local quaternion rotations.

For quaternion filtering, remember that q and $-q$ represent the same rotation. Before filtering quaternions, ensure sign continuity between consecutive frames. After filtering, normalize each quaternion again before converting it back to Euler angles for visualization.

Submit your Python code and answer the following questions:

1. What impact does filtering have on the motion?
2. Which artifacts occur?
3. How do the artifacts differ between position filtering, Euler-angle filtering, and quaternion filtering?

Assignment 2 (*Spacetime Constraint Method, 15 Points*)

For this task, consider the following situation:

You work as a technician on a film project in a large animation studio. Your boss enters your office visibly annoyed and complains about the arrogant, overpaid, and apparently not very capable actors. One performer was not able to hold his arm a few centimeters higher during a simple walking motion. In the final film, however, an animated baby dragon is supposed to walk underneath the actor's hand and later become the hero of the story. Since your boss still trusts your technical abilities, your task is to take the recorded walking motion and edit it so that the right hand is held higher.

The handout provides a frame-wise constrained optimization framework. The function

```
optimizeWithConstraint(skel, motions, constraint, joint_names, hand_name)
```

optimizes the selected joints so that the specified hand joint follows a desired target trajectory. The target trajectory is given as a $3 \times n$ array, where n is the number of motion frames.

A simple example for modifying AMC motion frames directly is:

```
for frame in motions:
    if "rhand" in frame:
        frame["rhand"][2] += 5.0
```

The joint names and numbers of our skeletons are shown in Figure 1. However, directly changing a single joint channel is not sufficient for producing a plausible motion. Instead, you should use the provided optimization framework and complete the objective function.

The concrete tasks are:

1. Complete the objective function `objfun(...)`. It should minimize the distance between the right hand and the target trajectory while keeping the optimized joint rotations close to the original motion.
2. Try different weightings and combinations of terms in the objective function. How does the result change when the hand-position term is weighted more strongly? How does the result change when the pose-preservation term is weighted more strongly?
3. Normally, one could optimize many skeletal joints. In this assignment, test different joint selections using the parameter `joint_names`. For example, compare optimizing only the upper arm with optimizing a larger part of the right arm.
4. Describe the visual results of the different configurations. Which joints produce plausible motion edits? Which configurations produce artifacts? Explain why.
5. How could you add a smoothness condition that prevents the arm from flipping back and forth between consecutive frames? You do not need to implement this term, but describe which additional error term you would add to the objective function.

Notes

- The assignments are due Monday, June 8th, at 23:59.
- Presenting the sheets is mandatory. If you cannot attend your scheduled time, contact your tutor and arrange another time or an online presentation.
- Upload a ZIP file to eCampus containing your solution so that it runs "out of the box". Name the ZIP file according to your last names.
- In case you have any questions about the assignments, please contact Melissa Steininger (s29mstei@uni-bonn.de)

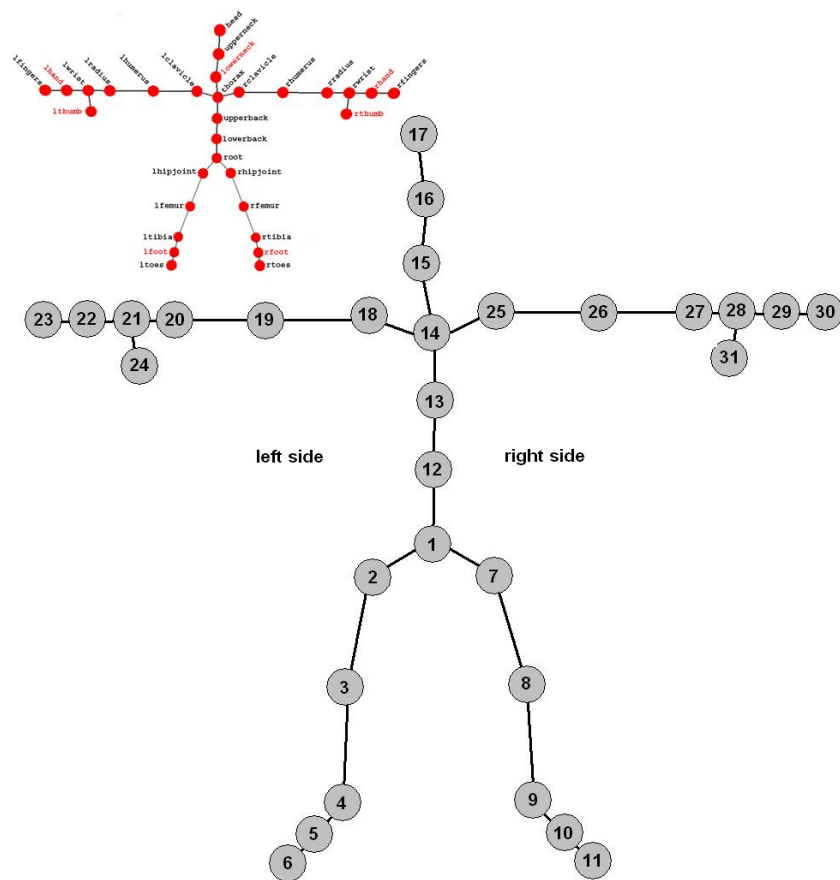


Figure 1: Skeletal joint names and numbers.